LAPORAN PEMOGRAMAN BERORIENTASI OBJEK
PEKAN 5 TRANSAKSI DINAMIS (POLYMORPHISM)

Disusun Oleh :

Dika Giowanda

NIM**:**

2311533025

Dosen Pengampu :

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Muhammad Dawi Syauqi

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2026

A. TAHAPAN DAN TUGAS PELAKSANAAN PRAKTIKUM

1. Praktikum Pekan 1(Class, Object, Constructor, dan Method)

A. Konsep Yang Dipelajari

- Class adalah cetak biru (blueprint) yang mendefinisikan data dan perilaku. Pada praktikum, class Rekening dibuat public agar bisa diakses dari luar class.
- Object adalah wujud nyata dari class. Objek dibuat dengan kata kunci new dan disimpan dalam variabel referensi (akunAktif).
- Atribut (state) adalah variabel yang menyimpan data objek: nomorRekening, namaPemilik, dan saldo.
- Constructor adalah method khusus bernama sama dengan class, otomatis dipanggil saat objek dibuat, dan dipakai untuk mengisi nilai awal atribut.
- Method (behavior) adalah tindakan yang bisa dilakukan objek, misalnya setor tunai dan cek informasi.

B. Kode Program

a. Class Rekening

Class Rekening berisi tiga atribut, satu constructor, dan dua method. Constructor menerima nilai dari parameter lalu menyalinnya ke atribut, kemudian mencetak pesan bahwa rekening berhasil dibuat.

```
7⊖    public Rekening(String nomor, String nama, double saldoAwal) {
8         nomorRekening = nomor;
9         namaPemilik = nama;
10        saldo = saldoAwal;
11        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
12    }
13
```

- setorTunai(double nominal) menambah saldo hanya jika nominal > 0. Jika tidak, ditampilkan pesan gagal. Ini contoh validasi sederhana di dalam method agar saldo tidak rusak oleh input yang tidak masuk akal.
- cekInformasi() mencetak nomor rekening, nama pemilik, dan saldo akhir.

```
    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }
```

Gambar kode secara lengkap:

```
package Pekan1_2511533025;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("----------------------");
    }
}
```

b. Class Main

- Inisialisasi: Rekening akunAktif = null; berarti variabel sudah ada tetapi belum menunjuk ke objek apa pun. Selain itu disiapkan Scanner untuk membaca input dan boolean isRunning sebagai penentu apakah menu masih berjalan.
- Perulangan while (isRunning): menampilkan menu terus-menerus sampai pengguna memilih keluar. Setelah nextInt() selalu dipanggil nextLine() untuk membersihkan sisa karakter enter di buffer, kalau tidak input String berikutnya akan terlewat.
- switch-case: memproses pilihan menu.
- Case 1 (buka rekening) membaca nomor, nama, saldo awal, lalu menjalankan akunAktif = new Rekening(no, nama, saldo). Baris inilah proses instansiasi sekaligus pemanggilan constructor.
- Case 2 (setor tunai) mengecek akunAktif == null terlebih dahulu. Pengecekan ini mencegah NullPointerException bila pengguna belum membuat rekening.
- Case 3 (tarik tunai) masih berupa tugas mandiri.
- Case 4 memanggil akunAktif.cekInformasi().

- Case 0 mengubah isRunning menjadi false sehingga perulangan berhenti, lalu Scanner ditutup dengan input.close().

Gambar kode secara lengkap:

```java
package Pekan1_2511533025;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; // Objek belum diinisialisasi (null)
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();

                    // Instansiasi Object / Menjalankan Constructor
                    akunAktif = new Rekening(no, nama, saldo);
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor); // Memanggil Behavior / method
                    }
                    break;

                case 3:
                    System.out.println("Fitur ini akan kerjakan sebagai Tugas Mandiri.");
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka rekening!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 0:
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima kasih!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

c. Tugas Praktikum

- Fitur Tarik Tunai (class Tabungan dan Play). Ditambahkan method tarikTunai() dengan dua aturan: nominal minimal Rp10.000 dan saldo harus mencukupi. Urutan if–else if–else dipakai agar aturan diperiksa bertahap; saldo baru dikurangi bila semua aturan lolos.

```
    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }
```

```
                case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal tarik tunai: ");
                        double tarik = input.nextDouble();
                        akunAktif.tarikTunai(tarik);
                    }
                    break;
```

- Tantangan Multi-Akun (class Simpanan dan Mulai). Satu variabel akunAktif tidak cukup untuk banyak nasabah, sehingga dipakai ArrayList<Simpanan> daftarRekening. Rekening baru otomatis menjadi akun aktif, nama akun aktif ditampilkan di atas menu, dan menu 5 (Ganti Akun) menelusuri daftar lalu mencocokkan nomor rekening dengan equals() untuk memilih akun lain.

```
                case 5: // Implementasi Ganti Akun
                    if (daftarRekening.isEmpty()) {
                        System.out.println("Error: Belum ada rekening yang terdaftar di sistem.");
                    } else {
                        System.out.println("\n--- DAFTAR REKENING TERSEDIA ---");
                        for (int i = 0; i < daftarRekening.size(); i++) {
                            Rekening2 r = daftarRekening.get(i);
                            System.out.println((i + 1) + ". No: " + r.nomorRekening + " | Nama: " + r.namaPemilik);
                        }
                        System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
                        String cariNo = input.nextLine();

                        boolean ditemukan = false;
                        for (Rekening2 r : daftarRekening) {
                            if (r.nomorRekening.equals(cariNo)) {
                                akunAktif = r;
                                System.out.println("Berhasil beralih ke rekening atas nama: " + akunAktif.namaPemilik);
                                ditemukan = true;
                                break;
                            }
                        }
                        if (!ditemukan) {
                            System.out.println("Error: Nomor rekening tidak ditemukan!");
                        }
                    }
                    break;
```

d. Gambar kode tugas secara lengkap:

1. Class Rekening

```java
package TugasPekan1_2511533025;
public class Rekening1 {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening1(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("----------------------");
    }
}
```

2. Class Main

```java
package TugasPekan1_2511533025;
import java.util.ArrayList;

public class Main2 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Menggunakan ArrayList untuk menampung banyak objek Rekening
        ArrayList<Rekening2> daftarRekening = new ArrayList<>();
        Rekening2 akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===");

        while (isRunning) {
            // Menampilkan informasi akun yang sedang aktif saat ini
            System.out.println("\n----------------------------------------");
            if (akunAktif != null) {
                System.out.println("Akun Aktif : " + akunAktif.namaPemilik + " (" + akunAktif.nomorRekening + ")");
            } else {
                System.out.println("Akun Aktif : Tidak ada akun yang dipilih");
            }
            System.out.println("----------------------------------------");

            System.out.println("Menu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun (Pilih Akun)"); // Menu Bonus / Tambahan
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();
                    input.nextLine(); // Membersihkan buffer

                    // Membuat objek baru dan menambahkannya ke ArrayList
                    Rekening2 rekeningBaru = new Rekening2(no, nama, saldo);
                    daftarRekening.add(rekeningBaru);

                    // Otomatis jadikan akun yang baru dibuat sebagai akun aktif
                    akunAktif = rekeningBaru;
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki atau memilih nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
                    break;

                case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki atau memilih nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal tarik tunai: ");
                        double tarik = input.nextDouble();
                        akunAktif.tarikTunai(tarik);
                    }
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka atau memilih rekening!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 5: // Implementasi Ganti Akun
                    if (daftarRekening.isEmpty()) {
                        System.out.println("Error: Belum ada rekening yang terdaftar di sistem.");
                    } else {
                        System.out.println("\n--- DAFTAR REKENING TERSEDIA ---");
                        for (int i = 0; i < daftarRekening.size(); i++) {
                            Rekening2 r = daftarRekening.get(i);
                            System.out.println((i + 1) + ". No: " + r.nomorRekening + " | Nama: " + r.namaPemilik);
                        }
                        System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
                        String cariNo = input.nextLine();

                        boolean ditemukan = false;
                        for (Rekening2 r : daftarRekening) {
                            if (r.nomorRekening.equals(cariNo)) {
                                akunAktif = r;
                                System.out.println("Berhasil beralih ke rekening atas nama: " + akunAktif.namaPemilik);
                                ditemukan = true;
                                break;
                            }
                        }
                        if (!ditemukan) {
                            System.out.println("Error: Nomor rekening tidak ditemukan!");
                        }
                    }
                    break;

                case 0:
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima kasih!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

e. Gambar Contoh Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 3025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 100000
Rekening atas nama Dika berhasil dibuat dengan saldo Rp100000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik tunai: 5000
Transaksi Gagal : Minimal nominal penarikan 10.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik tunai: 150000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp100000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik tunai: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp50000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 3025
Nama Pemilik : Dika
Saldo Akhir  : Rp50000.0
----------------------

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

----------------------------------------
Akun Aktif : Tidak ada akun yang dipilih
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 3025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Rekening atas nama Dika berhasil dibuat dengan saldo Rp500000.0

----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 3023
Masukkan Nama Pemilik: Fajar
Masukkan Saldo Awal: 1000000
Rekening atas nama Fajar berhasil dibuat dengan saldo Rp1000000.0

----------------------------------------
Akun Aktif : Fajar (3023)
----------------------------------------
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 3023
Nama Pemilik : Fajar
Saldo Akhir  : Rp1000000.0
----------------------

----------------------------------------
Akun Aktif : Fajar (3023)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 5

--- DAFTAR REKENING TERSEDIA ---
1. No: 3025 | Nama: Dika
2. No: 3023 | Nama: Fajar
Masukkan Nomor Rekening yang ingin diaktifkan: 3025
Berhasil beralih ke rekening atas nama: Dika

----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 3025
Nama Pemilik : Dika
Saldo Akhir  : Rp500000.0
----------------------

----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

2. Praktikum Pekan 2(Relasi Antar Objek(Class Diagram & Asosiasi)

A. Konsep Yang Dipelajari

- Class Diagram adalah diagram UML yang menggambarkan struktur sistem: class, atribut, method, dan hubungan antar objek. Penerjemahannya ke Java: class menjadi class, atribut menjadi variabel anggota (lengkap dengan tipe dan hak akses), method menjadi fungsi di dalam class.
- Asosiasi One-to-Many (1..*) berarti satu objek berhubungan dengan banyak objek kelas lain, misalnya satu Dosen mengajar banyak Mahasiswa. Implementasinya: class "One" menyimpan koleksi objek class "Many" sebagai atribut.
- ArrayList (java.util.ArrayList) adalah koleksi berukuran dinamis. Method utamanya add() untuk menambah, remove() untuk menghapus, dan get(indeks) untuk mengambil. Atribut tidak harus bertipe primitive, tipe class buatan sendiri juga boleh.

B. Kode Program Praktikum

a. Class Transaksi

punya atribut idTransaksi, jenis (Debit untuk penarikan, Kredit untuk penyetoran), dan nominal. Method cetakDetail() mencetak satu baris detail, sedangkan toString() yang di-override membuat objek otomatis tercetak rapi ketika langsung dipanggil dengan System.out.println(trx). Gambar kode secara lengkap:

```java
package Pekan2_2511533025;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;
    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
    @Override
    public String toString() {
        return "ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal;
    }

}
```

b. Class Rekening

Sekarang memiliki atribut ArrayList<Transaksi> riwayatTransaksi. Inilah wujud asosiasi 1-to-many: satu rekening menampung banyak transaksi. menginisialisasi this.riwayatTransaksi = new ArrayList<>(); di dalam constructor. Jika dilewati, list bernilai null dan saat add() dipanggil akan muncul NullPointerException. Pencatatan transaksi otomatis: setiap setor/tarik yang berhasil membuat objek Transaksi baru dan memasukkannya ke list. Perubahan dari materi dasar ke tugas Pekan 2 mencakup beberapa bagian berikut:

- Penambahan Import & Atribut Relasi
  Menambahkan import java.util.ArrayList; di bagian paling atas.
  Menambahkan atribut baruSystem.currentTimeMilliswayatTransaksi; untuk menampung daftar transaksi (relasi 1-to-many).
- Pembaruan Constructor
  Menambahkan inisialisasi this.riwayatTransaksi = new ArrayList<>(); di dalam constructor agar list siap digunakan dan tidak menyebabkan NullPointerExceptions.
- Pencatatan Riwayat pada setorTunai
  Saat setoran berhasil, program membuat objek transaksi baru dengan jenis "Kredit" dan id bernomor acak/waktu ("TRX-S-" + System.currentTimeMillis()), lalu menyimpannya ke riwayatTransaksi.

Gambar kode secara lengkap:

```java
package Pekan2_2511533025;

import java.util.ArrayList;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi Asosiasi (1-to-many): Satu rekening memiliki banyak transaksi
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;

            // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            // Merekam riwayat transaksi tarik tunai (Debit)
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("---------------------");
    }
}
```

Untuk Class main saya buat di tugas praktikum karena perubahan kode Class tersebut berada di bagian tugas.

C. Tugas Praktikum

a. Tugas 1 - Integrasi Rekam Jejak Penarikan. Penarikan yang berhasil harus dicatat sebagai Transaksi berjenis Debit dengan ID berawalan TRX-T-, sedangkan penarikan gagal tidak boleh masuk riwayat. Pada tugas ini nama class diubah agar kode praktikum tetap utuh: Transaksi menjadi Transaksi1, Rekening menjadi Rekening1, dan main menjadi MainPerbankan1. Nama constructor ikut diubah karena aturan Java mewajibkan nama constructor sama dengan nama class.

Menambahkan method tarikTunai(double nominal).

Menambahkan validasi penarikan minimal Rp10.000 dan cek kecukupan saldo.

Saat penarikan berhasil, program membuat objek transaksi baru dengan jenis "Debit" dan id ("TRX-T-" + System.currentTimeMillis()), lalu menyimpannya ke riwayatTransaksi.

Penambahan Method cetakMutasi (Tugas 2)

Menambahkan method baru public void cetakMutasi().

Menggunakan perulangan for-each (for (Transaksi trx : riwayatTransaksi)) untuk memanggil trx.cetakDetail() pada setiap riwayat.

Menampilkan pesan "Belum ada transaksi pada rekening ini" jika riwayatTransaksi masih kosong.

```
39      public void tarikTunai(double nominal) {
40          if (nominal < 10000) {
41              System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
42          } else if (nominal > saldo) {
43              System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
44          } else {
45              saldo -= nominal;
46
47              // Merekam riwayat transaksi tarik tunai (Debit)
48              String idTrx = "TRX-T-" + System.currentTimeMillis();
49              Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
50              riwayatTransaksi.add(trxBaru);
51
52              System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
53          }
54      }
55
```

b. Tugas 2 – Fitur Cetak Mutasi. Ditambahkan method cetakMutasi() pada class rekening (diberi nama Rekening2, dengan Transaksi2 sebagai class transaksinya, dan main bernama MainPerbankan2). Method ini mencetak header rekening, lalu: jika riwayat kosong tampil "Belum ada transaksi pada rekening ini", jika ada maka for-each memanggil cetakDetail() setiap transaksi. Fitur diakses melalui menu 6. Penambahan Fitur Ganti Akun (Menu 5)

Menambahkan opsi "5. Ganti Akun Aktif" pada tampilan menu utama.

Menambahkan case 5 pada struktur switch-case untuk mencari nomor rekening menggunakan perulangan for-each di dalam ArrayList<Rekening>.

Jika nomor rekening ditemukan, variabel akunAktif diubah ke objek rekening tersebut.

Penambahan Fitur Cetak Mutasi (Menu 6 - Tugas 2)

Menambahkan opsi "6. Cetak Mutasi (Riwayat)" pada daftar menu utama.

Menambahkan case 6 pada struktur switch-case untuk memanggil method akunAktif.cetakMutasi() setelah memastikan akunAktif tidak bernilai null.

```
        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun Aktif");
            System.out.println("6. Cetak Mutasi (Riwayat)"); // Gambar 3 (Tugas 2)[cite: 4]
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

                case 6:
                    // Gambar 3 (Tugas 2): Memanggil method cetakMutasi()[cite: 4]
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memilih rekening!");
                    } else {
                        akunAktif.cetakMutasi(); //[cite: 4]
                    }
                    break;
```

Gambar kode secara lengkap:

```
package TugasPekan2_2511533025;

import java.util.ArrayList;
import java.util.Scanner;

public class MainPerbankan2 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening2> daftarRekening = new ArrayList<>();
        Rekening2 akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun Aktif");
            System.out.println("6. Cetak Mutasi (Riwayat)"); // Gambar 3 (Tugas 2)[cite: 4]
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();

                    Rekening2 rekeningBaru = new Rekening2(no, nama, saldo);
                    daftarRekening.add(rekeningBaru);
                    akunAktif = rekeningBaru;
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memilih rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
                    break;

                case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memilih rekening!");
                    } else {
                        System.out.print("Masukkan nominal tarik: ");
                        double tarik = input.nextDouble();
                        akunAktif.tarikTunai(tarik);
                    }
                    break;
```

```
                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memilih rekening!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 5:
                    if (daftarRekening.isEmpty()) {
                        System.out.println("Error: Belum ada rekening yang terdaftar!");
                    } else {
                        System.out.print("Masukkan No Rekening yang ingin diaktifkan: ");
                        String cariNo = input.nextLine();
                        boolean ditemukan = false;

                        for (Rekening2 rek : daftarRekening) {
                            if (rek.nomorRekening.equalsIgnoreCase(cariNo)) {
                                akunAktif = rek;
                                ditemukan = true;
                                System.out.println("Akun aktif berhasil diubah ke atas nama: " + akunAktif.namaPemilik);
                                break;
                            }
                        }

                        if (!ditemukan) {
                            System.out.println("Error: Nomor rekening tidak ditemukan!");
                        }
                    }
                    break;

                case 6:
                    // Gambar 3 (Tugas 2): Memanggil method cetakMutasi()[cite: 4]
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memilih rekening!");
                    } else {
                        akunAktif.cetakMutasi(); //[cite: 4]
                    }
                    break;

                case 0:
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima kasih!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

D. Gambar Contoh Output

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----------------------------------------
Akun Aktif : Tidak ada akun yang dipilih
-----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 3025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Rekening atas nama Dika berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----------------------------------------
Akun Aktif : Dika (3025)
-----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6

===== MUTASI REKENING =====
No. Rekening : 3025
Nama Pemilik : Dika
---------------------------
Belum ada transaksi pada rekening ini
===========================
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor tunai: 200000
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp700000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik tunai: 100000
Tarik tunai Rp100000.0 berhasil. Saldo saat ini: Rp600000.0
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (3025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6

===== MUTASI REKENING =====
No. Rekening : 3025
Nama Pemilik : Dika
--------------------------
ID: TRX-S-1790093892567 | Jenis: Kredit | Nominal: Rp200000.0
ID: TRX-T-1790093907698 | Jenis: Debit | Nominal: Rp100000.0
==========================
```

3. Praktikum Pekan 3(Keamanan & Integritas Data(Enkapsulasi))

A. Kosenp Yang Dipelajari

Pada pekan sebelumnya atribut tidak memakai modifier sehingga bisa diubah langsung dari luar, misalnya akunAktif.saldo = 9999999. Sistem tidak punya cara mencegah manipulasi seperti ini. Enkapsulasi menyelesaikan masalah tersebut.

- Information Hiding (Robert C. Martin, Clean Code): sembunyikan detail internal, objek sebaiknya mengekspos behavior (method), bukan data (variabel).
- Access Modifier:

  private paling ketat; hanya bisa diakses class itu sendiri. Aturan emas untuk melindungi atribut.

  protected dapat diakses class dalam package yang sama dan subclass, berguna pada pewarisan.

  public dapat diakses dari mana saja; biasanya untuk method layanan.
- Getter dan Setter (Oracle, 2023): akses ke variabel private lewat method public. Keuntungannya, kita bisa menyisipkan validasi (misalnya cek panjang PIN) sebelum data diproses. Jika setter tidak disediakan, data bersifat read-only.

B. Kode Program Praktikum

a. Class Transaksi

Semua atribut dijadikan private dan hanya disediakan getter (getIdTransaksi, getJenis, getNominal) tanpa setter. Akibatnya riwayat transaksi tidak dapat diubah setelah objek dibuat, yang penting untuk integritas catatan keuangan.

```
3  public class Transaksi {
4      // 1. Mengubah semua atribut menjadi private
5      private String idTransaksi;
6      private String jenis;
7      private double nominal;
8
```

Gambar kode secara lengkap:

```java
package Pekan3_2511533025;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

    @Override
    public String toString() {
        return "ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal;
    }
}
```

b. Class Rekening

Pada atribut nomorRekening, namaPemilik, saldo, riwayatTransaksi, dan pin kini private. Saldo hanya bisa berubah lewat setorTunai() dan tarikTunai(), sehingga seluruh aturan validasi pasti dilewati. Validasi PIN di constructor dimana Constructor menerima parameter pinAwal. PIN hanya diterima bila panjangnya tepat 6 digit; jika tidak, sistem mencetak peringatan dan memakai PIN default "123456".

Seluruh atribut dikunci menjadi private untuk menyembunyikan data sensitif dari luar class. Constructor menerima tambahan parameter pinAwal dan dilengkapi validasi panjang PIN (harus 6 digit). Ditambahkan method Getter untuk membaca data private dan method otentikasi() untuk pencocokan PIN. Gambar kode secara lengkap:

```java
package TugasPekan3_2511533025;
import java.util.ArrayList;

public class Rekening3 {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif!

    // Implementasi Asosiasi (1-to-many): Satu rekening memiliki banyak transaksi
    private ArrayList<Transaksi3> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening3(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        }

        // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // Getter untuk mengakses riwayat transaksi dari luar kelas
    public ArrayList<Transaksi3> getRiwayatTransaksi() {
        return this.riwayatTransaksi;
    }

    // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;

            // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi3 trxBaru = new Transaksi3(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            // Merekam riwayat transaksi tarik tunai (Debit)
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi3 trxBaru = new Transaksi3(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("---------------------");
    }
}
```

Kode program MainPerbankan tidak saya masukan di sini karena terhubung dengan tugas praktikum.

C. Tugas Praktikum

Mengamankan CLI dengan Otentikasi

Menu 1: meminta PIN (String 6 digit) sebelum objek diinstansiasi, lalu meneruskannya ke new Rekening(no, nama, saldoAwal, pin).

Menu 3 dan Menu 6: sebelum method dijalankan, panggil akunAktif.otentikasi(pinInput). Jika true transaksi diproses; jika false dibatalkan dengan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

MainPerbankan. Tidak lagi memanggil atribut langsung (akunAktif.namaPemilik), melainkan getter (akunAktif.getNamaPemilik()). Pada case 1 program meminta PIN 6 digit dan meneruskannya ke constructor.

Perlindungan PIN pada menu: tarik tunai (menu 3), ganti akun (menu 5, memakai PIN akun target), dan riwayat transaksi (menu 6) meminta PIN lebih dahulu, sehingga orang lain tidak bisa berpindah atau melihat data akun tanpa otorisasi.

Keamanan Transaksi pada Menu 3 (Tarik Tunai)

Sebelum Tugas:

Setelah mengecek akunAktif tidak null, sistem langsung meminta masukan nominal tarik dan memanggil akunAktif.tarikTunai(tarik) tanpa meverifikasi PIN.

Setelah Tugas:

Sistem meminta input PIN terlebih dahulu dan memvalidasinya menggunakan akunAktif.otentikasi(pinInput). Transaksi hanya diproses jika hasil otentikasi bernilai true. Jika false, sistem mencetak "Akses Ditolak: PIN yang Anda masukkan salah!".

```
66              case 3:
67                  if (akunAktif == null) {
68                      System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
69                  } else {
70                      System.out.print("Masukkan PIN Anda: ");
71                      String pinInput = scanner.nextLine();
72
73                      // Memanggil fungsi otentikasi
74                      if (akunAktif.otentikasi(pinInput)) {
75                          System.out.print("Masukkan nominal tarik tunai: ");
76                          double tarik = scanner.nextDouble();
77                          akunAktif.tarikTunai(tarik);
78                      } else {
79                          System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
80                      }
81                  }
82                  break;
83
```

Keamanan Transaksi pada Menu 6 (Cetak Mutasi)

Sebelum Tugas:

Sistem langsung menampilkan mutasi transaksi dengan memanggil akunAktif.cetakMutasi() tanpa adanya proteksi PIN.

Setelah Tugas:

Sistem mengharuskan pengguna memasukkan PIN dan memverifikasinya melalui akunAktif.otentikasi(pinInput) sebelum mencetak mutasi. Jika PIN salah, akses ditolak.

```
                case 6:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan PIN Anda: ");
                        String inputPin = scanner.nextLine();
                        if (akunAktif.otentikasi(inputPin)) {
                            if (akunAktif.getRiwayatTransaksi().isEmpty()) { // Menggunakan getter agar sesuai encapsulation
                                System.out.println("Belum ada riwayat transaksi pada akun ini.");
                            } else {
                                System.out.println("--- RIWAYAT TRANSAKSI ---");
                                for (Object trx : akunAktif.getRiwayatTransaksi()) { // Atau gunakan tipe objek Transaksi jika ada kelasnya
                                    System.out.println(trx); // Memanggil representasi transaksi
                                }
                                System.out.println("-----------------------------------");
                            }
                        } else {
                            System.out.println("Akses Ditolak: PIN salah!");
                        }
                    }
                    break;
```

Gambar kode secara lengkap:

```
package TugasPekan3_2511533025;
import java.util.ArrayList;
import java.util.Scanner;
public class MainPerbankan3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Rekening3> daftarRekening = new ArrayList<>();
        Rekening3 akunAktif = null;
        int pilihan;
        do {
            System.out.println("\n=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===");
            System.out.println("-----------------------------------------");
            if (akunAktif == null) {
                System.out.println("Akun Aktif : Tidak ada akun yang dipilih");
            } else {
                System.out.println("Akun Aktif : " + akunAktif.getNamaPemilik() + " (" + akunAktif.getNomorRekening() + ")");
            }
            System.out.println("-----------------------------------------");
            System.out.println("Menu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun (Pilih Akun)");
            System.out.println("6. Tampilkan Riwayat Transaksi (Cetak Mutasi)");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            pilihan = scanner.nextInt();
            scanner.nextLine(); // Membersihkan buffer
            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No. Rekening: ");
                    String no = scanner.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = scanner.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldoAwal = scanner.nextDouble();
                    scanner.nextLine(); // Clear buffer

                    // Minta user menginputkan PIN (String angka 6 digit) sebelum instansiasi
                    System.out.print("Masukkan PIN (6 digit): ");
                    String pin = scanner.nextLine();

                    Rekening3 rekBaru = new Rekening3(no, nama, saldoAwal, pin);
                    daftarRekening.add(rekBaru);
                    akunAktif = rekBaru; // Otomatis jadikan akun aktif
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan nominal setor tunai: ");
                        double setor = scanner.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
                    break;
                // ==================== TUGAS POIN 2 (Menu 3) ====================
                case 3:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan PIN Anda: ");
                        String pinInput = scanner.nextLine();

                        // Memanggil fungsi otentikasi
                        if (akunAktif.otentikasi(pinInput)) {
                            System.out.print("Masukkan nominal tarik tunai: ");
                            double tarik = scanner.nextDouble();
                            akunAktif.tarikTunai(tarik);
                        } else {
                            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
                        }
                    }
```

```
                    }
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 5:
                    if (daftarRekening.isEmpty()) {
                        System.out.println("Belum ada rekening yang terdaftar.");
                    } else {
                        System.out.println("--- DAFTAR AKUN ---");
                        for (int i = 0; i < daftarRekening.size(); i++) {
                            System.out.println((i + 1) + ". " + daftarRekening.get(i).getNamaPemilik() + " (" + daftarRekening.get(i).getNomorRekening() + ")");
                        }
                        System.out.print("Pilih nomor akun: ");
                        int indeks = scanner.nextInt() - 1;
                        scanner.nextLine(); // Clear buffer

                        if (indeks >= 0 && indeks < daftarRekening.size()) {
                            Rekening3 target = daftarRekening.get(indeks);
                            System.out.print("Masukkan PIN akun target: ");
                            String pinAkses = scanner.nextLine();

                            if (target.otentikasi(pinAkses)) {
                                akunAktif = target;
                                System.out.println("Berhasil beralih ke akun " + akunAktif.getNamaPemilik());
                            } else {
                                System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
                            }
                        } else {
                            System.out.println("Pilihan tidak valid.");
                        }
                    }
                    break;

                // ================== TUGAS POIN 2 (Menu 6) ==================
                case 6:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan PIN Anda: ");
                        String inputPin = scanner.nextLine();
                        if (akunAktif.otentikasi(inputPin)) {
                            if (akunAktif.getRiwayatTransaksi().isEmpty()) { // Menggunakan getter agar sesuai encapsulation
                                System.out.println("Belum ada riwayat transaksi pada akun ini.");
                            } else {
                                System.out.println("--- RIWAYAT TRANSAKSI ---");
                                for (Object trx : akunAktif.getRiwayatTransaksi()) { // Atau gunakan tipe objek Transaksi jika ada kelasnya
                                    System.out.println(trx); // Memanggil representasi transaksi
                                }
                                System.out.println("-------------------------------");
                            }
                        } else {
                            System.out.println("Akses Ditolak: PIN salah!");
                        }
                    }
                    break;

                case 0:
                    System.out.println("Terima kasih telah menggunakan sistem perbankan.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid. Silakan coba lagi.");
            }
        } while (pilihan != 0);

        scanner.close();
    }
}
```

D. Gambar Contoh Output

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Tidak ada akun yang dipilih
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511533025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Masukkan PIN (6 digit): 013579
Rekening atas nama Dika berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Dika (2511533025)
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 3
Masukkan PIN Anda: 123456
Akses Ditolak: PIN yang Anda masukkan salah!

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Dika (2511533025)
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 3
Masukkan PIN Anda: 013579
Masukkan nominal tarik tunai: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp450000.0
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Dika (2511533025)
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 6
Masukkan PIN Anda: 123456
Akses Ditolak: PIN salah!

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Dika (2511533025)
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 6
Masukkan PIN Anda: 013579
--- RIWAYAT TRANSAKSI ---
ID: TRX-T-1790699037579 | Jenis: Debit | Nominal: Rp50000.0
-----------------------------------

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
--------------------------------------------
Akun Aktif : Dika (2511533025)
--------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi (Cetak Mutasi)
0. Keluar
Pilih menu: 0
Terima kasih telah menggunakan sistem perbankan.
```

4. Praktikum Pekan 4(Ekspansi Produk Keuangan(Inheritance))

A. Konsep Yang Dipelajari

Bank memiliki lebih dari satu jenis rekening: Rekening Tabungan (mendapat bunga bulanan) dan Rekening Giro (punya fasilitas overdraft/pinjaman). Membuat class baru dari nol untuk tiap jenis akan mendinstanceofogika PIN dan mutasi, melanggar prinsip DRY (Don't Repeat Yourself). Solusinya adalah pewarisan.

- Inheritance (Deitel & Deitel, 2017): class baru dibuat dari atribut dan behavior class yang sudah ada lalu ditambah kemampuan baru. Hubungannya disebut "Is-A": Tabungan is-a Rekening.
- extends dan super. extends membuat subclass mewarisi class induk. super(...) memanggil constructor induk dan wajib berada di baris pertama constructor anak. super.namaMethod() memanggil method asli induk yang mungkin sudah di-override.
- protected. Agar subclass bisa mengubah saldo dan riwayatTransaksi tanpa membukanya ke publik, atribut tersebut dibuat protected.
- Upcasting (implisit, aman). Menyimpan objek anak ke variabel induk: Rekening akun = new RekeningTabungan(...). Berguna agar berbagai jenis rekening bisa disimpan dalam satu ArrayList<Rekening>.
- Downcasting (eksplisit, berisiko). RekeningTabungan tab = (RekeningTabungan) akun; Bila objek aslinya bukan Tabungan, terjadi ClassCastException. Karena itu downcasting wajib didahului pengecekan instanceof.

B. Kode Program Praktikum

a. Class Transaksi

Tidak ada yang berubah.

```java
package Pekan4_2511533025;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

    @Override
    public String toString() {
        return "ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal;
    }
}
```

b. Class Rekening

Pada atribut nomorRekening, namaPemilik, dan pin tetap private; saldo dan riwayatTransaksi diubah menjadi protected. Konstruktor, getter, otentikasi, setorTunai, tarikTunai, dan cekInformasi tetap sama seperti pekan 3.

```java
public class Rekening {
    // 1. Atribut private
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

    // Gunakan protected agar Subclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

```

Gambar kode secara lengkap:

```java
package Pekan4_2511533025;
import java.util.ArrayList;

public class Rekening {
    // 1. Atribut private
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

    // Gunakan protected agar Subclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // Constructor
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        // Inisialisasi ArrayList
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    // Getter
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    public ArrayList<Transaksi> getRiwayatTransaksi() {
        return this.riwayatTransaksi;
    }

    // Otentikasi Internal
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;

            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("---------------------");
    }
}
```

c. Class RekeningTabungan

Class RekeningTabungan extends Rekening. Menambah atribut private double sukuBunga. Constructor menerima lima parameter (nomor, nama, saldoAwal, pinAwal, sukuBunga): empat nilai pertama diteruskan lewat super(...) dan sukuBunga disimpan sendiri.

Method tambahBungaAkhirBulan() menghitung bunga dari saldo saat ini, menambahkannya ke saldo (boleh langsung karena saldo protected), lalu mencatat Transaksi berjenis "Bunga" dengan ID berawalan TRX-B-.

```java
package Pekan4_2511533025;

public class RekeningTabungan extends Rekening {

    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        // Menghitung bunga
        // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
        // Karena atribut 'saldo' di class Rekening menggunakan modifier 'protected'
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

d. Class RekeningGiro

Class RekeningGiro extends Rekening. Menambah atribut private double batasOverdraft (batas maksimal cerukan/limit pinjaman). Constructor juga lima parameter dengan pola super(...) yang sama, ditambah getter getBatasOverdraft() agar nilainya dapat dibaca dari luar tanpa bisa diubah.

```java
package Pekan4_2511533025;

public class RekeningGiro extends Rekening {

    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        // Memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // (Catatan: Penarikan hingga limit overdraft akan kita selesaikan di Modul 5)
}
```

Kode program MainPerbankan tidak saya masukan di sini karena terhubung dengan tugas praktikum.

C. Tugas Praktikum

Pembuatan Multi-Produk (case 1). Setelah memasukkan nomor, nama, saldo, dan PIN, pengguna diminta memilih "1. Tinstanceofum | 2. Giro Bisnis". Variabel Rekening rekBaru = null disiapkan sebagai referensi bertipe induk.

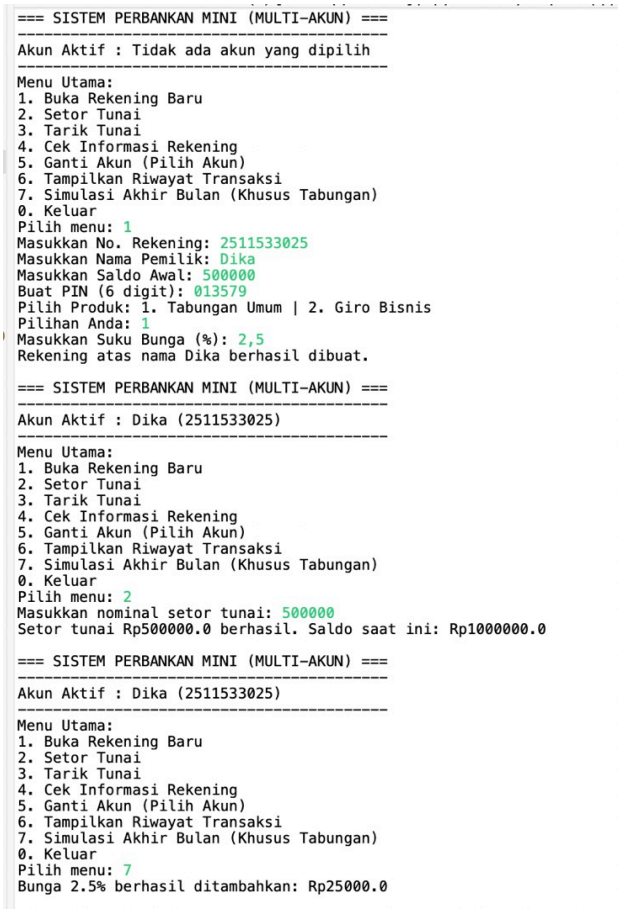
- Pilihan 1: meminta suku bunga (%), lalu new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga).
- Pilihan 2: meminta batas overdraft, lalu new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft).
- Pilihan lain: tampil pesan tidak valid dan pembuatan dibatalkan dengan break.

- Objek lalu dimasukkan ke ArrayList<Rekening> (upcasting) dan dijadikan akunAktif.
- Simulasi Waktu Akhir Bulan (case 7). Alurnya: (1) jika akunAktif == null, tampil peringatan; (2) uji akunAktif instanceof RekeningTabungan; (3) bila benar, downcast dan panggil tambahBungaAkhirBulan(); (4) bila bukan (misalnya RekeningGiro), tampil "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."
- Fitur bunga hanya ada di RekeningTabungan, bukan di Rekening. Karena akunAktif bertipe Rekening, kompiler tidak mengenali method tersebut sebelum objek di-downcast.

```
36              case 1:
37                  // Modifikasi Case 1: Buka Rekening Baru (Tugas 1)
38                  System.out.print("Masukkan No. Rekening: ");
39                  String no = scanner.nextLine();
40                  System.out.print("Masukkan Nama Pemilik: ");
41                  String nama = scanner.nextLine();
42                  System.out.print("Masukkan Saldo Awal: ");
43                  double saldoAwal = scanner.nextDouble();
44                  scanner.nextLine(); // Clear buffer
45                  System.out.print("Buat PIN (6 digit): ");
46                  String pin = scanner.nextLine();
47
48                  // 1. Opsi Produk
49                  System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
50                  System.out.print("Pilihan Anda: ");
51                  int jenisProduk = scanner.nextInt();
52                  scanner.nextLine(); // Clear buffer
53
54                  Rekening rekBaru = null;
55
56                  if (jenisProduk == 1) {
57                      // 2. Instansiasi RekeningTabungan
58                      System.out.print("Masukkan Suku Bunga (%): ");
59                      double sukuBunga = scanner.nextDouble();
60                      scanner.nextLine();
61                      rekBaru = new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga);
62                  } else if (jenisProduk == 2) {
63                      // 3. Instansiasi RekeningGiro
64                      System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
65                      double batasOverdraft = scanner.nextDouble();
66                      scanner.nextLine();
67                      rekBaru = new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft);
68                  } else {
69                      System.out.println("Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
70                      break;
71                  }
72
73                  // 4. Upcasting: Menyimpan ke daftarRekening bertipe ArrayList<Rekening>
74                  daftarRekening.add(rekBaru);
75                  akunAktif = rekBaru; // Otomatis jadikan akun aktif
76                  break;
```

```
163             case 7:
164                 // Simulasi Waktu Akhir Bulan (Tugas 2)
165                 if (akunAktif == null) {
166                     System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
167                 } else {
168                     // 2. Memeriksa tipe objek dengan instanceof
169                     if (akunAktif instanceof RekeningTabungan) {
170                         // 3. Downcasting ke tipe RekeningTabungan & memanggil method
171                         RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
172                         tabungan.tambahBungaAkhirBulan();
173                     } else {
174                         // 4. Pesan penolakan jika bukan RekeningTabungan
175                         System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
176                     }
177                 }
178                 break;
179
```

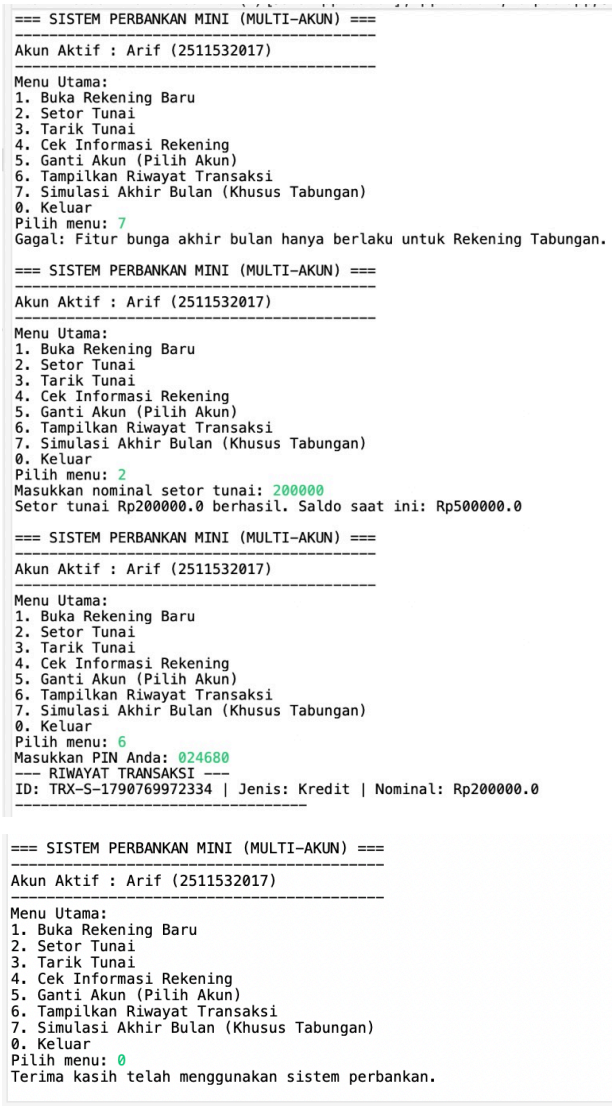
D. Gambar Contoh Output



```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
------------------------------------------
Akun Aktif : Tidak ada akun yang dipilih
------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511533025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Buat PIN (6 digit): 013579
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan Anda: 1
Masukkan Suku Bunga (%): 2,5
Rekening atas nama Dika berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
------------------------------------------
Akun Aktif : Dika (2511533025)
------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 2
Masukkan nominal setor tunai: 500000
Setor tunai Rp500000.0 berhasil. Saldo saat ini: Rp1000000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
------------------------------------------
Akun Aktif : Dika (2511533025)
------------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 2.5% berhasil ditambahkan: Rp25000.0
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2511533025
Nama Pemilik : Dika
Saldo Akhir  : Rp1025000.0
----------------------

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 3
Masukkan PIN Anda: 013579
Masukkan nominal tarik tunai: 25000
Tarik tunai Rp25000.0 berhasil. Saldo saat ini: Rp1000000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 6
Masukkan PIN Anda: 013579
--- RIWAYAT TRANSAKSI ---
ID: TRX-S-1790768769905 | Jenis: Kredit | Nominal: Rp500000.0
ID: TRX-B-1790768773495 | Jenis: Bunga | Nominal: Rp25000.0
ID: TRX-T-1790768801596 | Jenis: Debit | Nominal: Rp25000.0
----------------------------------
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511532017
Masukkan Nama Pemilik: Arif
Masukkan Saldo Awal: 300000
Buat PIN (6 digit): 024680
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan Anda: 2
Masukkan Batas Overdraft (Limit Pinjaman): 200000
Rekening atas nama Arif berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Arif (2511532017)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2511532017
Nama Pemilik : Arif
Saldo Akhir  : Rp300000.0
----------------------
```



a. Tahapan Pelaksanaan Praktikum Pekan 5

1. Konsep Yang Dipelajari

   Kata polymorphism berasal dari bahasa Yunani yang berarti "memiliki banyak bentuk". Dalam PBO, satu nama method dapat dieksekusi dengan cara berbeda-beda tergantung tipe objek aslinya saat program berjalan (runtime).

   - Method Overriding (Runtime Polymorphism). Menurut Eckel (2006), subclass yang mendefinisikan ulang method induk dengan nama dan parameter yang persis sama sedang melakukan overriding. Jika objek

dibuat dari kelas anak, Java selalu menjalankan versi milik kelas anak.

- Dynamic Method Dispatch (Late Binding). Menurut Oracle (2023), JVM memutuskan method mana yang dipanggil saat program berjalan, bukan saat kompilasi. Ketika akunAktif.tarikTunai() dipanggil, JVM melihat isi akunAktif (Tabungan atau Giro) lalu menjalankan logika yang sesuai tanpa if-else atau switch-case.
- Anotasi @Override. Praktik terbaik (Sierra dan Bates, 2005) untuk memberi tahu compiler bahwa method memang bermaksud menimpa method induk. Jika nama method salah ketik, compiler akan menampilkan error.
- Parameter polimorfik. Method yang menerima parameter bertipe induk (Rekening) dapat diberi objek dari subclass mana pun (Tabungan atau Giro) tanpa perlu tahu tipe persisnya.

2. Kode Program

a) Class Transaksi

Tidak berubah.

```java
package Pekan5_2511533025;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

    @Override
    public String toString() {
        return "ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal;
    }
}
```

b) Class Rekening

Perubahan kelas ini berhubungan dengan tugas, jadi saya jelaskan nanti di bagian tugas praktikum.

```java
package Pekan5_2511533025;

import java.util.ArrayList;

public class Rekening {
    // Atribut private
    private String nomorRekening;
    private String namaPemilik;
    private String pin; // Data sensitif!

    // Protected agar Subclass bisa mengaksesnya
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // Constructor
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }
        // Inisialisasi ArrayList
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }
    // Getter
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    public ArrayList<Transaksi> getRiwayatTransaksi() {
        return this.riwayatTransaksi;
    }

    // Otentikasi Internal
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;

            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }
```

```java
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
            return false;
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
            return false;
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
            return true;
        }
    }

    // ========================================================================
    // BAGIAN YANG DITAMBAHKAN (Latihan/Tugas: Fitur Transfer Antar Rekening)
    // ========================================================================
    public void transfer(Rekening tujuan, double nominal, String pinInput) {
        if (!otentikasi(pinInput)) {
            System.out.println("Transfer Gagal: PIN yang Anda masukkan salah!");
            return;
        }
        boolean suksesTarik = this.tarikTunai(nominal);
        if (suksesTarik) {
            tujuan.setorTunai(nominal);
            System.out.println("Transfer sebesar Rp" + nominal + " ke rekening " + tujuan.getNomorRekening() + " (" + tujuan.getNamaPemilik() + ") berhasil!");
        } else {
            System.out.println("Transfer Gagal: Proses penarikan dari saldo asal tidak berhasil.");
        }
    }
    // ========================================================================

    public void cekInformasi() {
        System.out.println("----- INFO REKENING -----");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("-------------------------");
    }
}
```

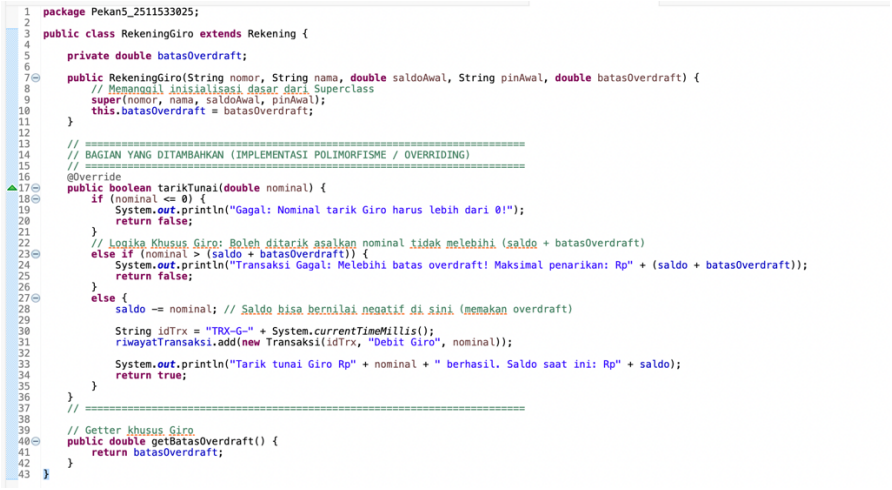
c) Class RekeningTabungan

Tidak ada perubahan

```java
package Pekan5_2511533025;

public class RekeningTabungan extends Rekening {

    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        // Menghitung bunga
        // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
        // Karena atribut 'saldo' di class Rekening menggunakan modifier 'protected'
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

d) Class RekeningGiro

Perubahan kelas ini berhubungan dengan tugas, jadi saya jelaskan nanti di bagian tugas praktikum.



```java
package Pekan5_2511533025;

public class RekeningGiro extends Rekening {

    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        // Memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // ======================================================================
    // BAGIAN YANG DITAMBAHKAN (IMPLEMENTASI POLIMORFISME / OVERRIDING)
    // ======================================================================
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
            return false;
        }
        // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }
        else {
            saldo -= nominal; // Saldo bisa bernilai negatif di sini (memakan overdraft)

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }
    // ======================================================================

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

e) Class MainPerbankan

Perubahan kelas ini berhubungan dengan tugas, jadi saya jelaskan nanti di bagian tugas praktikum.

```java
package Pekan5_2511533025;

import java.util.ArrayList;

public class MainPerbankan {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        int pilihan;

        do {
            System.out.println("\n=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===");
            System.out.println("-----------------------------------------");
            if (akunAktif == null) {
                System.out.println("Akun Aktif : Tidak ada akun yang dipilih");
            } else {
                System.out.println("Akun Aktif : " + akunAktif.getNamaPemilik() + " (" + akunAktif.getNomorRekening() + ")");
            }
            System.out.println("-----------------------------------------");
            System.out.println("Menu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun (Pilih Akun)");
            System.out.println("6. Tampilkan Riwayat Transaksi");
            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
            // --- MODIFIKASI 1: Penambahan Tampilan Menu 8 ---
            System.out.println("8. Transfer ke Rekening Lain");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            pilihan = scanner.nextInt();
            scanner.nextLine(); // Clear buffer

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No. Rekening: ");
                    String no = scanner.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = scanner.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldoAwal = scanner.nextDouble();
                    scanner.nextLine(); // Clear buffer
                    System.out.print("Buat PIN (6 digit): ");
                    String pin = scanner.nextLine();

                    System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
                    System.out.print("Pilihan Anda: ");
                    int jenisProduk = scanner.nextInt();
                    scanner.nextLine(); // Clear buffer
```

```
                    Rekening rekBaru = null;

                    if (jenisProduk == 1) {
                        System.out.print("Masukkan Suku Bunga (%): ");
                        double sukuBunga = scanner.nextDouble();
                        scanner.nextLine();
                        rekBaru = new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga);
                    } else if (jenisProduk == 2) {
                        System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
                        double batasOverdraft = scanner.nextDouble();
                        scanner.nextLine();
                        rekBaru = new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft);
                    } else {
                        System.out.println("Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
                        break;
                    }

                    daftarRekening.add(rekBaru);
                    akunAktif = rekBaru;
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan nominal setor tunai: ");
                        double setor = scanner.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
                    break;

                case 3:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan PIN Anda: ");
                        String inputPin = scanner.nextLine();
                        if (akunAktif.otentikasi(inputPin)) {
                            System.out.print("Masukkan nominal tarik tunai: ");
                            double tarik = scanner.nextDouble();
                            akunAktif.tarikTunai(tarik);
                        } else {
                            System.out.println("Transaksi Gagal: PIN salah!");
                        }
                    }
                    break;
```

```
                case 4:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 5:
                    if (daftarRekening.isEmpty()) {
                        System.out.println("Belum ada rekening yang terdaftar.");
                    } else {
                        System.out.println("--- DAFTAR AKUN ---");
                        for (int i = 0; i < daftarRekening.size(); i++) {
                            System.out.println((i + 1) + ". " + daftarRekening.get(i).getNamaPemilik() + " (" + daftarRekening.get(i).getNomorRekening() + ")");
                        }
                        System.out.print("Pilih nomor akun: ");
                        int indeks = scanner.nextInt() - 1;
                        scanner.nextLine(); // Clear buffer

                        if (indeks >= 0 && indeks < daftarRekening.size()) {
                            Rekening target = daftarRekening.get(indeks);
                            System.out.print("Masukkan PIN akun target: ");
                            String pinAkses = scanner.nextLine();

                            if (target.otentikasi(pinAkses)) {
                                akunAktif = target;
                                System.out.println("Berhasil beralih ke akun " + akunAktif.getNamaPemilik());
                            } else {
                                System.out.println("Gagal beralih: PIN salah!");
                            }
                        } else {
                            System.out.println("Pilihan tidak valid.");
                        }
                    }
                    break;

                case 6:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan PIN Anda: ");
                        String inputPin = scanner.nextLine();
                        if (akunAktif.otentikasi(inputPin)) {
                            if (akunAktif.getRiwayatTransaksi().isEmpty()) {
                                System.out.println("Belum ada riwayat transaksi pada akun ini.");
                            } else {
                                System.out.println("--- RIWAYAT TRANSAKSI ---");
                                for (Transaksi trx : akunAktif.getRiwayatTransaksi()) {
                                    System.out.println(trx);
                                }
                                System.out.println("-------------------------------------");
                            }
                        } else {
                            System.out.println("Akses Ditolak: PIN salah!");
                        }
                    }
                    break;
```

```
                case 7:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        if (akunAktif instanceof RekeningTabungan) {
                            RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
                            tabungan.tambahBungaAkhirBulan();
                        } else {
                            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
                        }
                    }
                    break;

                // ========================================================================
                // MODIFIKASI 2: Penambahan Case 8 (Fitur Transfer Antar Rekening)
                // ========================================================================
                case 8:
                    if (akunAktif == null) {
                        System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
                    } else {
                        System.out.print("Masukkan Nomor Rekening Tujuan: ");
                        String noTujuan = scanner.nextLine();
                        if (noTujuan.equalsIgnoreCase(akunAktif.getNomorRekening())) {
                            System.out.println("Gagal: Tidak dapat melakukan transfer ke rekening sendiri!");
                            break;
                        }
                        Rekening akunTujuan = null;
                        for (Rekening rek : daftarRekening) {
                            if (rek.getNomorRekening().equalsIgnoreCase(noTujuan)) {
                                akunTujuan = rek;
                                break;
                            }
                        }
                        if (akunTujuan != null) {
                            System.out.print("Masukkan Nominal Transfer: Rp");
                            double nominalTrf = scanner.nextDouble();
                            scanner.nextLine(); // Clear buffer

                            System.out.print("Masukkan PIN Anda: ");
                            String pinTrf = scanner.nextLine();
                            akunAktif.transfer(akunTujuan, nominalTrf, pinTrf);
                        } else {
                            System.out.println("Gagal: Nomor rekening tujuan tidak ditemukan!");
                        }
                    }
                    break;

                case 0:
                    System.out.println("Terima kasih telah menggunakan sistem perbankan.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid. Silakan coba lagi.");
            }
        } while (pilihan != 0);

        scanner.close();
    }
}
```

b. Tugas Praktikum Pekan 5

1. Method transfer() di class Rekening. Signature: public void transfer(Rekening tujuan, double nominal, String pinInput). Parameter tujuan bertipe induk sehingga bisa menerima Tabungan maupun Giro. Alurnya: (1) otentikasi(pinInput), jika false batalkan; (2) panggil this.tarikTunai(nominal), jika this adalah Giro maka otomatis memakai aturan overdraft, (3) jika hasilnya true, panggil tujuan.setorTunai(nominal).
2. Integrasi ke CLI (menu 8). Menu "8. Transfer ke Rekening Lain" meminta nomor rekening tujuan, mencari nomor itu di daftarRekening, lalu bila ditemukan meminta nominal dan PIN dan memanggil akunAktif.transfer(akunTujuan, nominal, pin). Pada kode ditambahkan juga validasi: transfer ke rekening sendiri ditolak, dan pesan khusus bila nomor tujuan tidak ditemukan. Pencarian memakai for-each dan equalsIgnoreCase().

Penjelasan kode yang berubah:

a. Class Rekening

- Perubahan Tipe Kembalian (Return Type)

  Sebelumnya: public void tarikTunai(double nominal)

  Menjadi: public boolean tarikTunai(double nominal)

  Alasan: Agar method dapat mengembalikan status keberhasilan transaksi (true jika berhasil, false jika gagal). Nilai balik ini sangat berguna ketika method penarikan dipanggil oleh fitur lain seperti fitur transfer atau overriding pada subclass (RekeningGiro).

- Perubahan Kondisi Validasi Nominal Minimal

  Sebelumnya: if (nominal < 10000)

  Menjadi: if (nominal <= 0)

  Alasan: Menyesuaikan standar validasi pada modul agar menolak semua nominal ≤0.

- Penambahan Pernyataan return

  Menambahkan return false; di blok kondisi gagal (nominal ≤0 atau saldo tidak mencukupi).

  Menambahkan return true; di akhir blok else saat penarikan saldo berhasil.

```
46    public void setorTunai(double nominal) {
47        if (nominal > 0) {
48            saldo += nominal;
49
50            String idTrx = "TRX-S-" + System.currentTimeMillis();
51            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
52            riwayatTransaksi.add(trxBaru);
53            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
54        } else {
55            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
56        }
57    }
58
59    public boolean tarikTunai(double nominal) {
60        if (nominal <= 0) {
61            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
62            return false;
63        } else if (nominal > saldo) {
64            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
65            return false;
66        } else {
67            saldo -= nominal;
68
69            String idTrx = "TRX-T-" + System.currentTimeMillis();
70            riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));
71
72            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
73            return true;
74        }
75    }
76
```

- Parameter Polimorfik (Rekening tujuan, ...)

  Parameter tujuan bertipe Rekening (kelas induk).

  Artinya, objek penerima transfer bisa berupa instans dari Rekening biasa maupun RekeningGiro.

- Tahap 1 - Otentikasi PIN

  Menggunakan if (!otentikasi(pinInput)). Jika PIN salah, fungsi langsung keluar (return;) dan membatalkan proses transfer.
- Tahap 2 - Penggunaan Polimorfisme pada this.tarikTunai(nominal)

  Jika objek pengirim (this) adalah Rekening biasa, maka ia mengecek batasan saldo normal.

  Jika objek pengirim (this) adalah RekeningGiro, Java secara otomatis mengeksekusi method tarikTunai hasil @Override (yang memperbolehkan saldo minus hingga batas overdraft).
- Tahap 3 - Pengecekan Nilai Kembalian boolean

  tarikTunai mengembalikan nilai true jika penarikan saldo berhasil.

  Jika nilai kembaliannya true, maka perintah tujuan.setorTunai(nominal) baru akan dijalankan untuk menambah saldo rekening penerima.

```
77      // ================================================================
78      // BAGIAN YANG DITAMBAHKAN (Latihan/Tugas: Fitur Transfer Antar Rekening)
79      // ================================================================
80      public void transfer(Rekening tujuan, double nominal, String pinInput) {
81          if (!otentikasi(pinInput)) {
82              System.out.println("Transfer Gagal: PIN yang Anda masukkan salah!");
83              return;
84          }
85          boolean suksesTarik = this.tarikTunai(nominal);
86          if (suksesTarik) {
87              tujuan.setorTunai(nominal);
88              System.out.println("Transfer sebesar Rp" + nominal + " ke rekening " + tujuan.getNomorRekening() + " (" + tujuan.getNamaPemilik() + ") berhasil!");
89          } else {
90              System.out.println("Transfer Gagal: Proses penarikan dari saldo asal tidak berhasil.");
91          }
92      }
93      // ================================================================
```

b. Class RekeningGiro

- Penggunaan Anotasi @Override

  Menandakan bahwa method tarikTunai pada kelas RekeningGiro menimpa (menggantikan) method tarikTunai milik kelas induk (Rekening).
- Perubahan Logika Batas Penarikan (Overdraft)

  Pada Rekening biasa, penarikan maksimal hanya sampai batas saldo.

  Pada RekeningGiro, kondisi pengecekan diubah menjadi else if (nominal > (saldo + batasOverdraft)).

  Hal ini memungkinkan nasabah menarik uang melebihi saldo yang dimiliki, selama total penarikan masih dalam batas toleransi saldo + batasOverdraft. Saldo akhir nantinya bisa bernilai minus (negatif).

- Pencatatan Transaksi Khusus Giro

  ID transaksi menggunakan awalan "TRX-G-" dan tipe transaksi dicatat sebagai "Debit Giro" agar bisa dibedakan dengan transaksi rekening biasa pada riwayat transaksi.

```
16      @Override
17      public boolean tarikTunai(double nominal) {
18          if (nominal <= 0) {
19              System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
20              return false;
21          }
22          // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
23          else if (nominal > (saldo + batasOverdraft)) {
24              System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
25              return false;
26          }
27          else {
28              saldo -= nominal; // Saldo bisa bernilai negatif di sini (memakan overdraft)
29
30              String idTrx = "TRX-G-" + System.currentTimeMillis();
31              riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));
32
33              System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
34              return true;
35          }
36      }
```

c. Class MainPerbankan

- Penambahan Pilihan Menu pada Output CLI

  Mengatur cetakan layar menu dengan menambahkan baris System.out.println("8. Transfer ke Rekening Lain").

- Pengecekan Kondisi case 8

  Pemeriksaan Akun Aktif: Memastikan pengirim tidak null sebelum memproses transfer.

  Input Nomor Rekening Tujuan: Mengambil nomor rekening target dari input user.

  Pencarian Objek (Linear Search): Melakukan perulangan (for-each) pada daftarRekening untuk mencocokkan getNomorRekening().

  Validasi Keberadaan Rekening:

  Jika akunTujuan ditemukan, sistem meminta input nominal dan PIN.

  Selanjutnya memanggil fungsi akunAktif.transfer(akunTujuan, nominalTrf, pinTrf).

  Jika tidak ditemukan, menampilkan pesan error "Nomor rekening tujuan tidak ditemukan!".

```
177                 case 8:
178                     if (akunAktif == null) {
179                         System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
180                     } else {
181                         System.out.print("Masukkan Nomor Rekening Tujuan: ");
182                         String noTujuan = scanner.nextLine();
183                         if (noTujuan.equalsIgnoreCase(akunAktif.getNomorRekening())) {
184                             System.out.println("Gagal: Tidak dapat melakukan transfer ke rekening sendiri!");
185                             break;
186                         }
187                         Rekening akunTujuan = null;
188                         for (Rekening rek : daftarRekening) {
189                             if (rek.getNomorRekening().equalsIgnoreCase(noTujuan)) {
190                                 akunTujuan = rek;
191                                 break;
192                             }
193                         }
194                         if (akunTujuan != null) {
195                             System.out.print("Masukkan Nominal Transfer: Rp");
196                             double nominalTrf = scanner.nextDouble();
197                             scanner.nextLine(); // Clear buffer
198 
199                             System.out.print("Masukkan PIN Anda: ");
200                             String pinTrf = scanner.nextLine();
201                             akunAktif.transfer(akunTujuan, nominalTrf, pinTrf);
202                         } else {
203                             System.out.println("Gagal: Nomor rekening tujuan tidak ditemukan!");
204                         }
205                     }
206                     break;
```

c. Gambar Ouput Kode Program Praktikum

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----------------------------------------
Akun Aktif : Tidak ada akun yang dipilih
-----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511533025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Buat PIN (6 digit): 013579
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan Anda: 1
Masukkan Suku Bunga (%): 2,5
Rekening atas nama Dika berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----------------------------------------
Akun Aktif : Dika (2511533025)
-----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511533023
Masukkan Nama Pemilik: Fajar
Masukkan Saldo Awal: 100000
Buat PIN (6 digit): 024680
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan Anda: 2
Masukkan Batas Overdraft (Limit Pinjaman): 200000
Rekening atas nama Fajar berhasil dibuat.
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Fajar (2511533023)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 2511533023
Gagal: Tidak dapat melakukan transfer ke rekening sendiri!

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Fajar (2511533023)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 5
--- DAFTAR AKUN ---
1. Dika (2511533025)
2. Fajar (2511533023)
Pilih nomor akun: 1
Masukkan PIN akun target: 013579
Berhasil beralih ke akun Dika
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 2511533023
Masukkan Nominal Transfer: Rp150000
Masukkan PIN Anda: 013579
Tarik tunai Rp150000.0 berhasil. Sisa saldo: Rp350000.0
Setor tunai Rp150000.0 berhasil. Saldo saat ini: Rp250000.0
Transfer sebesar Rp150000.0 ke rekening 2511533023 (Fajar) berhasil!

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 2511533023
Masukkan Nominal Transfer: Rp50000
Masukkan PIN Anda: 000000
Transfer Gagal: PIN yang Anda masukkan salah!
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 2511533023
Masukkan Nominal Transfer: Rp1000000
Masukkan PIN Anda: 013579
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp350000.0
Transfer Gagal: Proses penarikan dari saldo asal tidak berhasil.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2511533025
Nama Pemilik : Dika
Saldo Akhir  : Rp350000.0
----------------------
```

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Dika (2511533025)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 5
--- DAFTAR AKUN ---
1. Dika (2511533025)
2. Fajar (2511533023)
Pilih nomor akun: 2
Masukkan PIN akun target: 024680
Berhasil beralih ke akun Fajar

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
----------------------------------------
Akun Aktif : Fajar (2511533023)
----------------------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2511533023
Nama Pemilik : Fajar
Saldo Akhir  : Rp250000.0
----------------------
```

C. Link Repository GitHub

1. Link Repository Kode Program
   https://github.com/DikaSiNpc/PrakPBO_C_2511533025/tree/PrakPBO_Pekan5_C_2511533025/src

2. Link Video Penjelasan Kode Program Pekan 1-5
   https://youtu.be/8TA1grgV1hA?si=mDZtHnxNhU-fL8-d